

WEEK 03 MODULE
Programming Practice

TOPIC : 1 QUICK SORT

b) Medium

que. 2 What is Sorting?

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort



Quick Sort

Definition : Quick Sort is a Divide and Conquer sorting algorithm.

1. Choose a pivot element.
2. Rearrange the array so that:
 1. Elements smaller than the pivot go to the left.
 2. Elements greater than the pivot go to the right.
3. Repeat the same process for the left and right subarrays until the entire array is sorted.

Key Points:-

- Uses Divide and Conquer.
- Selects a pivot element.
- Partitions the array around the pivot.
- Recursively sorts the left and right subarrays.
- In-place sorting algorithm (requires very little extra memory).
- Average Time Complexity: $O(n \log n)$.
- Worst Case Time Complexity: $O(n^2)$ (when poor pivots are repeatedly chosen).

How It Works:-

38 27 43 3 9 82 10

Choose Pivot = **38**

27 3 9 10 | 38 | 43 82

↓

Sort left part → **3 9 10 27**

Sort right part → **43 82**

↓

3 9 10 27 38 43 82

Problem Statement

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the Top 5 highest trade values.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

1. Question Analysis

A financial analytics company receives daily trade-value data from multiple stock exchanges in random order. To analyze high-value trading activity and identify market trends, the trade values must first be arranged in descending order. After sorting, the analysts need to identify the Top 5 highest trade values, calculate the average of these Top 5 values, and determine how many trade values are greater than the overall average of all trade values.

Since a large amount of trading data is processed every day, an efficient sorting algorithm is required. Therefore, the task is to implement the **Quick Sort** algorithm to sort the trade values in descending order without using any built-in sorting methods.

2. Problem Analysis

The given problem requires sorting an unsorted list of trade values in descending order using the Quick Sort algorithm. Since the financial analytics company receives trading data from multiple stock exchanges, the sorting technique should efficiently handle large datasets while maintaining good performance.

Quick Sort solves this problem by selecting a pivot element, partitioning the array into two parts based on the pivot, and recursively sorting each partition until the entire array is arranged in descending order. After sorting, the Top 5 highest trade values are identified, their average is calculated, and the overall average of all trade values is used to count the number of trade values that are greater than it. These operations help analysts identify significant trading activities and study market trends.

3. Algorithm Selection Analysis

Selected Algorithm: Quick Sort

- Quick Sort follows the Divide and Conquer technique.
- It selects a pivot element and partitions the array around it.
- It recursively sorts the left and right partitions until the entire array is sorted.
- It efficiently handles large datasets containing thousands of trade values.
- The sorted array makes it easy to identify the Top 5 highest trade values and perform further statistical analysis.
- It performs sorting in-place, requiring very little extra memory.

4. Points to Remember

- Quick Sort follows the Divide and Conquer technique.
- A pivot element is selected during each recursive call.
- The array is partitioned so that elements greater than the pivot are placed on one side and smaller elements on the other.
- This problem requires descending order, so larger trade values are placed before smaller trade values.
- The partitioning and recursive process continues until the entire array becomes sorted.

- After sorting, the Top 5 highest trade values are obtained from the beginning of the sorted array.
- The average of the Top 5 values is calculated.
- The overall average of all trade values is calculated.
- The number of trade values greater than the overall average is determined.
- Time Complexity
 - Best Case : $O(N \log N)$
 - Average Case : $O(N \log N)$
 - Worst Case : $O(N^2)$
- Space Complexity : $O(\log N)$ (due to recursive call stack)

5. Core Idea / Logic

- Select the last element of the current subarray as the pivot.
- Partition the array so that trade values greater than the pivot are placed on the left and smaller values on the right.
- Recursively apply Quick Sort to the left and right partitions until the complete array is sorted in descending order.
- Display the sorted trade values.
- Select the first five elements of the sorted array as the Top 5 highest trade values.
- Calculate the average of the Top 5 trade values.
- Calculate the overall average of all trade values.
- Count the number of trade values greater than the overall average.
- Display all the required results.

6. Execution Flow

Part A : Execution Flow

Input Phase

1. The program starts execution from the `main()` method.
2. The user enters the number of trade values (N).
3. The program validates whether N is within the specified range.
4. An array of size N is created to store the trade values.
5. The user enters all trade values.
6. Each trade value is validated and stored in the array.
7. The `quickSort()` method is called to sort the trade values in descending order.

Processing Phase

1. The `quickSort()` method selects the last element of the current subarray as the pivot.
2. The `partition()` method rearranges the trade values such that values greater than the pivot are placed on the left and smaller values on the right.
3. The pivot is placed at its correct position in the descending order.
4. The `quickSort()` method is recursively called for the left and right partitions until the complete array becomes sorted.
5. The first five elements of the sorted array are identified as the Top 5 highest trade values.
6. The average of the Top 5 trade values is calculated.
7. The overall average of all trade values is calculated.
8. The number of trade values greater than the overall average is counted.

Output Phase

1. Control returns to the `main()` method.
2. The sorted trade values are displayed in descending order.
3. The Top 5 highest trade values are displayed.

4. The average of the Top 5 trade values is displayed.
5. The count of trade values greater than the overall average is displayed.
6. The program terminates successfully.

Part B : Steps

Step 1: Start the execution of the program.

Step 2: Read the number of trade values (N) from the user.

Step 3: Validate the value of N. If N is less than 5 or greater than 100000, display an error message and terminate the program.

Step 4: Create an array of size N to store the trade values.

Step 5: Read each trade value from the user, validate that every value lies between 1 and 1000000000, store the valid values in the array, and simultaneously calculate the total sum of all trade values.

Step 6: Call the quickSort(arr, 0, n - 1) method to sort the trade values in descending order.

Step 7: Select the last element of the current subarray as the pivot and partition the array so that all trade values greater than or equal to the pivot are placed on the left and the remaining values are placed on the right.

Step 8: Place the pivot at its correct position and recursively apply Quick Sort to the left and right partitions until the complete array becomes sorted in descending order.

Step 9: Display the sorted trade values.

Step 10: Select the first five elements of the sorted array as the Top 5 highest trade values and calculate their total.

Step 11: Divide the total of the Top 5 trade values by 5 to calculate the average of the Top 5 values.

Step 12: Calculate the overall average of all trade values by dividing the total sum of all trade values by N.

Step 13: Traverse the sorted array and count the number of trade values that are greater than the overall average.

Step 14: Display the Top 5 trade values, their average, and the number of trade values greater than the overall average.

Step 15: Close the Scanner object and terminate the program.

7. Program Structure Analysis

i. Variable n

```
int n;
```

Purpose:

- Stores the total number of trade values entered by the user.
- Determines the size of the array.
- Defines the number of trade values to be sorted.

- Constraint:

$$5 \leq n \leq 100000$$

- Minimum value of n can be 5.
- Maximum value of n can be 100000.

ii. Array arr[]

```
long[] arr = new long[n];
```

Purpose:

- Stores all trade values entered by the user.
 - Acts as the main array on which Quick Sort is performed.
 - Stores the trade values in descending order after sorting is completed.
- Constraint:

$1 \leq \text{Trade Value} \leq 10^9$

- Minimum trade value can be 1.
- Maximum trade value can be 1000000000.

iii. Variable pivot

```
long pivot = arr[high];
```

Purpose:

- Stores the pivot element selected for partitioning.
- Uses the last element of the current subarray as the pivot.
- Helps divide the array into two partitions during Quick Sort.

iv. Variables low, high and pivotIndex

```
quickSort(arr, low, high);
int pivotIndex;
```

Purpose:

- low represents the starting index of the current subarray.
- high represents the ending index of the current subarray.
- pivotIndex stores the final position of the pivot after partitioning.
- These variables control the recursive sorting process.

v. Variables i and j

```
int i = low - 1;
for(int j = low; j < high; j++)
```

Purpose:

- i keeps track of the position where the next larger trade value should be placed.
- j traverses the current subarray and compares each element with the pivot.
- Together, they help partition the array in descending order.

vi. Variable temp

```
long temp;
```

Purpose:

- Temporarily stores an element during swapping.
- Prevents data loss while exchanging the positions of two trade values.

vii. Variable totalSum

```
long totalSum = 0;
```

Purpose:

- Stores the sum of all trade values entered by the user.
- Used to calculate the overall average of all trade values.
- Is updated while reading the input values.

viii. Variable top5Sum

```
long top5Sum = 0;
```

Purpose:

- Stores the sum of the Top 5 highest trade values.
- Used to calculate the average of the Top 5 trade values after sorting.

ix. Variable top5Average

```
double top5Average = top5Sum / 5.0;
```

Purpose:

- Stores the average of the Top 5 highest trade values.
- Uses 5.0 to perform floating-point division for accurate results.

x. Variable overallAverage

```
double overallAverage = totalSum / (double)n;
```

Purpose:

- Stores the average of all trade values entered by the user.
- Acts as the reference value for counting trade values greater than the average.

xi. Variable count

```
int count = 0;
```

Purpose:

- Counts the number of trade values that are greater than the overall average.
- Increases whenever a trade value satisfies the required condition.

xii. Input Loop

```
for(int i = 0; i < n; i++)  
{
```

```
arr[i] = sc.nextLong();  
}
```

Purpose:

- Accepts all trade values from the user.
- Validates every trade value according to the given constraints.
- Stores the values in the array.
- Simultaneously calculates the total sum of all trade values.

xiii. quickSort() Method

```
quickSort(arr, low, high);
```

Purpose:

- Recursively sorts the array in descending order.
- Calls the partition() method to position the pivot correctly.
- Continues sorting the left and right partitions until the complete array becomes sorted.

xiv. partition() Method

```
partition(arr, low, high);
```

Purpose:

- Selects the pivot element.
- Rearranges the elements so that larger values are placed before the pivot and smaller values after it.
- Returns the final position of the pivot.
- Forms the basis of the Quick Sort algorithm.

xv. Partition Loop

```
for(int j = low; j < high; j++)
```

Purpose:

- Traverses the current subarray.
- Compares every trade value with the pivot.
- Swaps elements whenever a larger trade value is found.
- Helps partition the array in descending order.

xvi. Top 5 Loop

```
for(int i = 0; i < 5; i++)
```

Purpose:

- Traverses the first five elements of the sorted array.
- Displays the Top 5 highest trade values.
- Simultaneously calculates their total.

xvii. Count Loop

```
for(int i = 0; i < n; i++)
```

Purpose:

- Traverses the sorted array.
- Compares every trade value with the overall average.
- Counts the number of trade values greater than the overall average.

xviii. Output Loop

```
for(int i = 0; i < n; i++)  
{  
    System.out.print(arr[i] + " ");  
}
```

Purpose:

- Traverses the sorted array.
- Displays all trade values in descending order.
- Produces the final sorted output before displaying the calculated statistical information.

8. Java Program

```
package week3_module;  
import java.util.Scanner;  
public class QuickSortMedium {  
  
    // Partition function for descending order  
    static int partition(long[] arr, int low, int high) {  
  
        long pivot = arr[high];  
        int i = low - 1;  
  
        for (int j = low; j < high; j++) {  
  
            // Arrange in descending order  
            if (arr[j] >= pivot) {  
  
                i++;  
  
                long temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
  
        long temp = arr[i + 1];  
        arr[i + 1] = arr[high];  
        arr[high] = temp;  
  
        return i + 1;  
    }  
  
    // Recursive Quick Sort  
    static void quickSort(long[] arr, int low, int high) {  
  
        if (low < high) {  
  
            int pivotIndex = partition(arr, low, high);  
  
            quickSort(arr, low, pivotIndex - 1);  
        }  
    }  
}
```



```

        quickSort(arr, pivotIndex + 1, high);
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of trade values : ");
    int n = sc.nextInt();

    if (n < 5 || n > 100000) {
        System.out.println("Error: N must be between 5 and 100000.");
        sc.close();
        return;
    }

    long[] arr = new long[n];
    long totalSum = 0;

    System.out.print("Enter the trade values : ");

    for (int i = 0; i < n; i++) {

        arr[i] = sc.nextLong();

        if (arr[i] < 1 || arr[i] > 1000000000L) {
            System.out.println("Error: Trade value must be between 1 and 1000000000.");
            sc.close();
            return;
        }

        totalSum += arr[i];
    }

    // Perform Quick Sort
    quickSort(arr, 0, n - 1);

    // Display sorted trade values
    System.out.print("Sorted Trade Values : ");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();

    // Display Top 5 values
    System.out.print("Top 5: ");

    long top5Sum = 0;

    for (int i = 0; i < 5; i++) {
        System.out.print(arr[i] + " ");
        top5Sum += arr[i];
    }
    System.out.println();

    // Average of Top 5

    double top5Average = top5Sum / 5.0;
    System.out.println("Average of Top 5: " + top5Average);

    // Overall average

```

```

        double overallAverage = totalSum / (double) n;

        // Count values above overall average
        int count = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] > overallAverage) {
                count++;
            }
        }

        System.out.println("Values Above Overall Average: " + count);

        sc.close();
    }
}

```

NOTE:- The constraint is: $1 \leq \text{Trade Value} \leq 10^9$

A single trade value fits in an int, but when adding thousands of trade values, the total sum can easily exceed the maximum value an int can hold (2,147,483,647). Using long for the array values and the sum prevents integer overflow and makes the program safe for the given constraints.

9. Program's output

Enter the number of trade values : 8

Enter the trade values : 500 1200 700 2000 900 1500 3000 2500

Sorted Trade Values : 3000 2500 2000 1500 1200 900 700 500

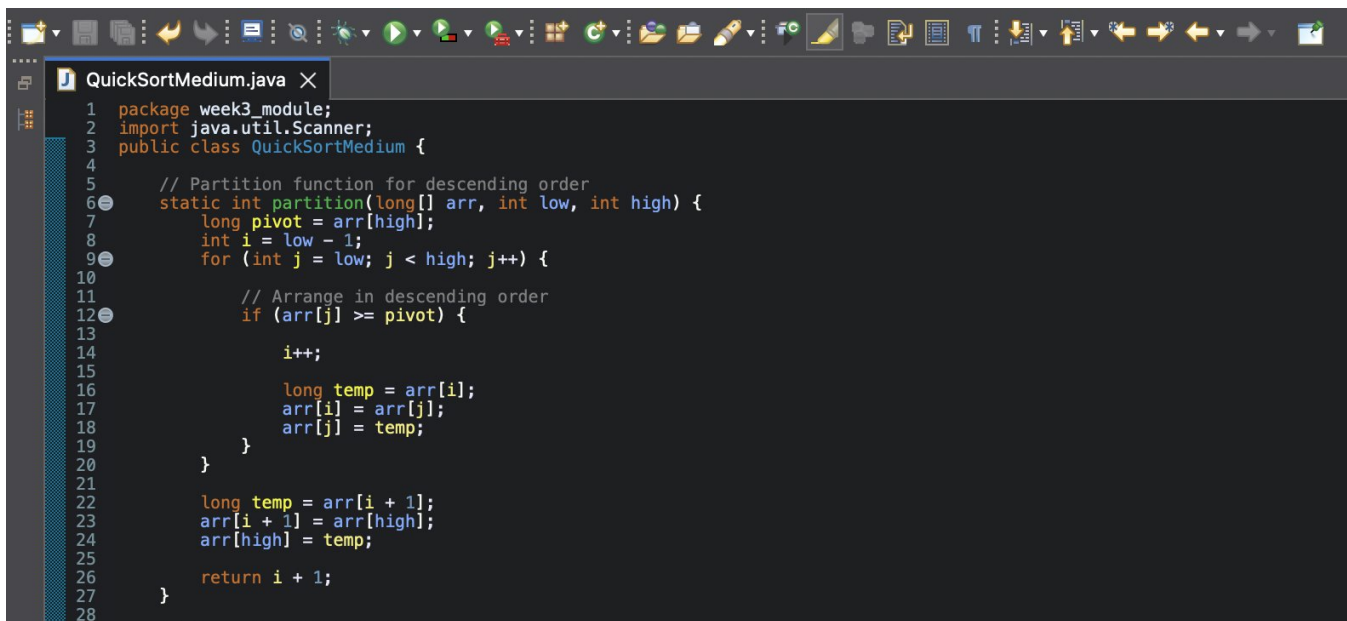
Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040.0

Values Above Overall Average: 3

10. Screenshot

Program



```

1 package week3_module;
2 import java.util.Scanner;
3 public class QuickSortMedium {
4
5     // Partition function for descending order
6     static int partition(long[] arr, int low, int high) {
7         long pivot = arr[high];
8         int i = low - 1;
9         for (int j = low; j < high; j++) {
10
11             // Arrange in descending order
12             if (arr[j] >= pivot) {
13
14                 i++;
15
16                 long temp = arr[i];
17                 arr[i] = arr[j];
18                 arr[j] = temp;
19             }
20         }
21
22         long temp = arr[i + 1];
23         arr[i + 1] = arr[high];
24         arr[high] = temp;
25
26         return i + 1;
27     }
28

```

```

29 // Recursive Quick Sort
30 static void quickSort(long[] arr, int low, int high) {
31
32     if (low < high) {
33
34         int pivotIndex = partition(arr, low, high);
35
36         quickSort(arr, low, pivotIndex - 1);

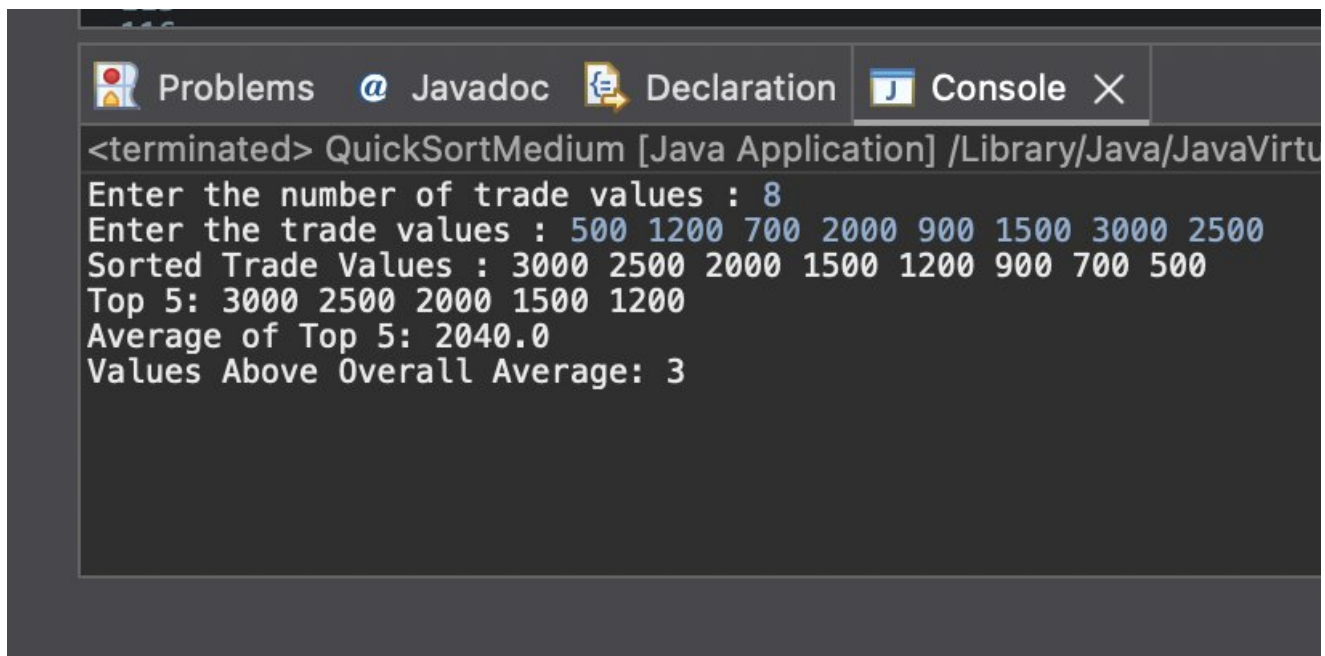
```

```

59     for (int i = 0; i < n; i++) {
60         arr[i] = sc.nextLong();
61
62         if (arr[i] < 1 || arr[i] > 1000000000L) {
63             System.out.println("Error: Trade value must be between 1 and 1000000000.");
64             sc.close();
65             return;
66         }
67
68         totalSum += arr[i];
69     }
70
71     // Perform Quick Sort
72     quickSort(arr, 0, n - 1);
73
74     // Display sorted trade values
75     System.out.print("Sorted Trade Values : ");
76     for (int i = 0; i < n; i++) {
77         System.out.print(arr[i] + " ");
78     }
79     System.out.println();
80
81     // Display Top 5 values
82     System.out.print("Top 5: ");
83
84     long top5Sum = 0;
85
86     for (int i = 0; i < 5; i++) {
87         System.out.print(arr[i] + " ");
88         top5Sum += arr[i];
89     }
90     System.out.println();
91
92     // Average of Top 5
93     double top5Average = top5Sum / 5.0;
94     System.out.println("Average of Top 5: " + top5Average);
95
96     // Overall average
97     double overallAverage = totalSum / (double) n;
98
99     // Count values above overall average
100    int count = 0;
101
102    for (int i = 0; i < n; i++) {
103        if (arr[i] > overallAverage) {
104            count++;
105        }
106    }
107
108    System.out.println("Values Above Overall Average: " + count);
109
110    sc.close();
111 }
112 }
113 }
114

```

Program's output



```
<terminated> QuickSortMedium [Java Application] /Library/Java/JavaVirtualMachine/...
Enter the number of trade values : 8
Enter the trade values : 500 1200 700 2000 900 1500 3000 2500
Sorted Trade Values : 3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040.0
Values Above Overall Average: 3
```

11. EXPLANTAION

Every day, different stock exchanges send trade values.

Example: 500 1200 700 2000 900 1500 3000 2500

These numbers represent the value of different trades. But they're completely random.

The analyst says,

- Arrange them from highest to lowest.
- the Top 5 trade values,
- their average,
- and how many trades are above the overall average."

STEP 1 — main() starts

```
public static void main(String[] args)
```

Every Java program starts here.

STEP 2 – Scanner

```
Scanner sc = new Scanner(System.in)
```

STEP 3 – Read N

```
int n = sc.nextInt();
n = 8
```

Meaning

There are 8 trade values coming next.

STEP 4 – Validate N

```
if(n<5 || n>100000)
```

Question: Why check this?

Because the question itself says $5 \leq N \leq 100000$

STEP 5 – Create Array

```
long arr[] = new long[n];  
n=8
```

Java creates **Eight** empty boxes.

These boxes will store the trade values.

STEP 6 – totalSum

```
long totalSum=0;
```

Question Why are we storing total sum?

Because later we need Overall Average.

Formula Average = Sum /Number of Elements

Without sum, average cannot be calculated.

STEP 7 – Input Loop

```
for(int i=0;i<n;i++)
```

whether user enters

8

500

50000

numbers.

Loop automatically repeats

N times.

- Inside loop

```
arr[i]=sc.nextLong();
```

Suppose input = 500

Stored = 500 -----

Next = 1200

Stored = 500 1200 _ _ _ _ _

Eventually = 500 1200 700 2000 900 1500 3000 2500

STEP 8 – Validate Trade Value

Immediately after reading,

```
if(arr[i]<1 || arr[i]>1000000000)
```

Question Why?

Because question says $1 \leq \text{Trade Value} \leq 10^9$

If someone enters = 500 = Invalid.

If someone enters = 2000000000 = Also invalid.

Program terminates.

STEP 9 – Add to totalSum

```
totalSum+=arr[i];
```

500 + 1200 + 700

Initially

totalSum=0

After 500, totalSum=500

After 1200, totalSum=1700

After 700, totalSum= 2400

Eventually totalSum= 12300

This helps us calculate overall average later.

STEP 10 – Call Quick Sort

```
quickSort(arr,0,n-1);
```

Current array 500 1200 700 2000 900 1500 3000 2500

Indexes	0	1	2	3	4	5	6	7
---------	---	---	---	---	---	---	---	---

Control leaves main() and enters quickSort().

STEP 11 – quickSort()

```
quickSort(arr,low,high)
```

Initially : low=0

high=7

Question What does Quick Sort do?

It repeatedly

1. chooses a Pivot
2. partitions
3. recursively sorts

STEP 12 – Pivot

Inside partition()

pivot=arr[high];

Question Why last element?

Because this implementation chooses last element as pivot.

Current array 500 1200 700 2000 900 1500 3000 2500

Pivot = 2500

STEP 13 – i

int i=low-1;

Initially low=0

i=-1

Question Why -1?

Because we haven't found any large element yet. It always tells where the next larger value should go.

STEP 14 – Variable j

for(int j = low; j < high; j++)

Question:

What is j?

Its job is to move from the left side of the array to the element before the pivot and inspect every trade value.

Current array: 500 1200 700 2000 900 1500 3000 2500 & Pivot = 2500

j starts from index 0 and keeps moving towards index 6.

STEP 15 – Comparison

Inside the loop:

```
if(arr[j] >= pivot)
```

NOTE : >= and **not** <=

Because the question wants descending order.

Descending means Largest
 ↓
 Smallest

So whenever we find a value greater than or equal to the pivot, it belongs on the left side.

Current array = 500 1200 700 2000 900 1500 3000 2500 & Pivot = 2500

First Comparison

500 >=2500? = No. -> Nothing happens.

Next 1200>=2500? = No. -> Nothing happens.

Next 700>=2500? = No. -> Nothing happens

Next 2000>=2500? = No. -> Nothing happens

Next 900>=2500? = No. -> Nothing happens

Next 1500>=2500? = No. -> Nothing happens

Next 3000>=2500? = YES! -> This value should come before the pivot.

STEP 16 – Increment i

Whenever the condition becomes true,

```
i++;
```

Initially i=-1

Now i=0

Why?

Because the first position of the "greater values" section is now index 0.

STEP 17 – Swapping

Now this code runs

```
long temp = arr[i];  
arr[i] = arr[j];  
arr[j] = temp;
```

Current array: 500 1200 700 2000 900 1500 3000 2500

We're swapping 500 and 3000

Result : 3000 1200 700 2000 900 1500 500 2500

Question: Why swap?

Because we want Greater values on Left side

STEP 18 – Pivot Placement

Once the loop finishes,

we execute

```
temp = arr[i+1];  
arr[i+1] = arr[high];  
arr[high] = temp;
```

Question Why?

Because now everything before *i* is greater than the pivot. So the pivot should come immediately after them.

Current array = 3000 1200 700 2000 900 1500 500 2500

i=0

Swap arr[1] and pivot

Result = 3000 2500 700 2000 900 1500 500 1200

Now 2500 is in its correct position.

STEP 19 – Return Pivot Index

```
return i+1;
```

Current *i*=0

Returns = 1

Meaning = 250 is permanently fixed at index 1 and will never move again.

STEP 20 – Recursion

Back inside quickSort() we do quickSort(arr, low, pivotIndex - 1);

This sorts = 3000

Nothing to do. Single element. Already sorted.

Then quickSort(arr, pivotIndex + 1, high);

Sorts = 700 2000 900 1500 500 1200

Exactly the same process repeats.

Choose Pivot

↓

Partition

↓

Recursively Sort

↓

Partition Again

↓

Recursively Sort

until every pivot reaches its correct position.

Finally = 3000 2500 2000 1500 1200 900 700 500

STEP 21 – Display Sorted Trade Values

```
for(int i=0;i<n;i++)
```

prints : 3000 2500 2000 1500 1200 900 700 500

STEP 22 – Top 5

```
for(int i=0;i<5;i++)
```

Question Why only first 5?

Because the array is already sorted in descending order. Largest values are automatically at the beginning.

Array : 3000 2500 2000 1500 1200 900 700 500

Top 5 : 3000 2500 2000 1500 1200

STEP 23 – top5Sum

Initially long top5Sum=0; As the loop runs Final = 10200

STEP 24 – Top 5 Average

```
double top5Average = top5Sum / 5.0;
```

STEP 25 – Overall Average

```
double overallAverage = totalSum / (double)n;
```

STEP 26 – Count Values Above Overall Average

Initially count=0;

Loop : for(int i=0;i<n;i++) checks every value.

3000 > 1537.5 ? = YES -> Count = 1

2500 = YES -> Count = 2

2000 = YES -> Count = 3

1500 > 1537.5 ? = NO -> No increment.

Final = 4

STEP 27 – Display Everything

Finally the program prints:

Sorted Trade Values: 3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Values Above Overall Average: 4

STEP 28 – Close Scanner

sc.close();

The program has finished taking input, so the Scanner object is closed to release the associated system resources.

The Entire Journey

Start



Read N



Validate N



Create Array



Read All Trade Values



Validate Each Trade Value



Calculate Total Sum



Call Quick Sort



Select Pivot



Partition the Array



Recursively Sort Left and Right Partitions



Obtain the Array in Descending Order



Display the Sorted Trade Values



Select the First Five Elements as the Top 5 Trade Values



Calculate the Average of the Top 5 Values



Calculate the Overall Average of All Trade Values



Count the Number of Trade Values Greater Than the Overall Average



Display All Results



Close Scanner



End Program

Quick Sort works by repeatedly:

1. Choosing a pivot,
2. Partitioning the array so the pivot lands in its correct position,
3. Recursively sorting the left and right partitions.

Once the array is sorted in descending order, everything else becomes straightforward:

- The Top 5 are simply the first five elements.
- The Top 5 average is their sum divided by 5.
- The overall average is the total sum divided by N .
- Counting values above the overall average is just a single traversal through the sorted array.